

ACADEMIC ATTENDANCE TRACKING SYSTEM

SECP3623 Database Programming | Section 1

Brendan Chia | Gui Kah Sin | Sabrina Heng | Lau Yan Kai

THE PROBLEM

Manual Inefficient

Current methods of tracking student attendance rely on physical sign-in sheets or fragmented spreadsheets. This leads to:

- ✓ **Data Redundancy:** Duplicate records across different departments.
- ✓ **High Latency:** Delay in detecting students with poor attendance.
- ✓ **Audit Failures:** Inability to verify historical records accurately.

OUR SOLUTION

We developed a centralised **Relational Database Management System (RDBMS)** designed for scalability and integrity.

Core Architecture

- ✓ **3NF Normalization**: Eliminated data anomalies by splitting entities.
- ✓ **6 Core Entities**: Students, Lecturers, Courses, Classes, Enrollments, Attendance.
- ✓ **Scalability**: Designed to handle thousands of daily check-ins.

DATABASE SCHEMA DESIGN

Core Entities

We designed 6 normalised tables to minimise redundancy:

- ✓ **Students:** Student information
- ✓ **Lecturers:** Lecturer information
- ✓ **Courses & Classes:** Academic structure.
- ✓ **Enrollments:** Junction table linking Students to Classes.
- ✓ **Attendance_Records:** Transactional data with timestamps.

DDL & INTEGRITY CONSTRAINTS

```
CREATE TABLE IF NOT EXISTS students
(
  student_id INT PRIMARY KEY
  AUTO_INCREMENT,
  matric_no VARCHAR(12) NOT NULL
  UNIQUE,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL
  UNIQUE,
  programme VARCHAR(100) NOT
  NULL
);
```

Structural Integrity

We utilised strict DDL commands to enforce business rules at the database level:

- ✓ **Primary Keys:** Ensured unique identification for every record.
- ✓ **Foreign Keys:** Enforced relationships between students and classes.
- ✓ **Cascading Actions:** Automated cleanup; if a student withdraws, their enrollments are removed automatically.

DML IMPLEMENTATION LOGIC

INSERTION

Populated the system with 10 students, 10 lecturers, and 14 classes using optimised multi-row INSERT statements.

```
INSERT INTO lecturers (lecturer_id, staff_n  
(3001, '10231', 'Dr. Azlan', 'Department of  
(3002, '12124', 'Dr. Siti', 'Department of S  
(3003, '9742', 'Dr. Lee', 'Department of Com  
(3004, '12111', 'Dr. Sean', 'Department of A  
(3005, '15946', 'Dr. Hani', 'Department of M  
(3006, '7963', 'Dr. Faridah', 'Department of  
(3007, '16508', 'Dr. Lim', 'Department of So  
(3008, '9832', 'Dr. Nurin', 'Department of M  
(3009, '13126', 'Dr. Nies', 'Department of C  
(3010, '7864', 'Dr. Najmi', 'Department of A
```

AUTOMATED UPDATES

Implemented logic to mark students as "Late".

UPDATE attendance SET
status='Late' WHERE time >
'09:00:00'

```
UPDATE attendance_records  
SET status = 'Late'  
WHERE attendance_time  
IS NOT NULL AND  
MINUTE(attendance_time) > 0;
```

CASCADING DELETES

Configured ON DELETE
CASCADE. Removing a student
automatically removes their
enrollment records, preventing
orphans.

```
ALTER TABLE enrollments  
ADD CONSTRAINT enrollments_ibfk_1  
FOREIGN KEY (student_id)  
REFERENCES students(student_id)  
ON DELETE CASCADE;
```

QUERYING

Aggregation & Grouping

We used queries to generate report metrics.

✓ **Grouping:** Used HAVING statements to analyse those lecturers who have more than 1 class per week.

✓ **Aggregation:** Calculated total student counts per program and compared them against global MAX/MIN values.

lecturer	total_classes
Dr. Azlan	2
Dr. Sean	2
Dr. Nurin	2
Dr. Nies	2

programme	total_students	student_status
Data Engineering	3	MAX
Computer Networks and Security	2	
Bioinformatics	2	
Software Engineering	2	
Graphics and Multimedia Software	1	MIN

JOINS & SUBQUERIES

CORRELATED SUBQUERY

Used to find the latest attendance date for each student by correlating the inner query with the outer student table.

student_id	name	last_attendance
1001	Aisyah	2025-11-11
1008	Brendan Chia	2025-11-11
1017	Fatin Afifah	2025-11-11
1023	Gui Kah Sin	2025-11-11
1056	Lau Yan Kai	2025-11-11
1080	Matthew	2025-11-11
1107	Muhammad Iqram	2025-11-11
1129	Sabrina Heng	2025-11-11
1212	Yusuf	2025-11-11
1213	Transaction Test Student	NULL

SELF JOIN

Identify pairs of different students who were present in class on the same day

student1	student2	attendance_date	status1	status2
1	3	2025-11-10	Present	Present
1	5	2025-11-10	Present	Present
1	7	2025-11-10	Present	Late
1	9	2025-11-10	Present	Present
1	11	2025-11-10	Present	Absent
1	13	2025-11-10	Present	Present
1	15	2025-11-10	Present	Present
1	17	2025-11-10	Present	Present
1	19	2025-11-10	Present	Present
3	5	2025-11-10	Present	Present
3	7	2025-11-10	Present	Late
3	9	2025-11-10	Present	Present

SET OPERATIONS

Used UNION to combine lists of students from 'Data Engineering' and 'Software Engineering' for departmental reports.

name	programme
Gui Kah Sin	Data Engineering
Matthew	Data Engineering
Sabrina Heng	Data Engineering
Lau Yan Kai	Software Engineering
Transaction Test Student	Software Engineering

TRANSACTION MANAGEMENT

```
START TRANSACTION;
```

```
INSERT INTO students (matric_no,  
name, email, programme)  
VALUES ('A23CS9000',  
'Transaction Test Student',  
'test@graduate.utm.my',  
'Software Engineering');
```

```
INSERT INTO enrollments (student_id,  
class_id)  
VALUES (LAST_INSERT_ID(), 1);
```

```
COMMIT;
```

✓ **Scenario:** Registering a new student AND immediately enrolling them in a class.

✓ **Result:** If the enrollment fails, the student creation is rolled back, preventing "ghost" students with no classes.

PERFORMANCE OPTIMISATION

We implemented BTREE indexes on dates.

BEFORE indexing

20

rows scanned (ALL)

AFTER indexing

1

rows scanned (REF)

```
CREATE INDEX idx_attendance_date ON attendance_records (attendance_date) USING BTREE;
```

PERFORMANCE OPTIMISATION

...and FULLTEXT indexes on names.

BEFORE indexing

10

rows scanned (ALL)

AFTER indexing

1

rows scanned (REF)

```
CREATE FULLTEXT INDEX idx_student_name ON students (name);
```

CHALLENGE & FUTURE WORK

Challenges Faced

- ✓ Writing complex Correlated Subqueries for statistical reports
- ✓ Ensuring Transaction atomicity during multi-table inserts.
- ✓ Analysing EXPLAIN plans to verify index usage.

Future Improvements

- ✓ **Stored Procedures:** Encapsulate complex logic for reuse.
- ✓ **Triggers:** Auto-update status to 'Late' upon insertion.
- ✓ **Frontend Interface:** Web interface for lecturers and administrators.

CONCLUSION

We made the system.

- ✓ built a reliable database
- ✓ data is safe and organised
- ✓ improved the system speed
- ✓ searching for records is instant
- ✓ automated attendance tracking
- ✓ eliminates manual errors

“

*The project was a **success**, as it **solves the manual tracking problem.***

”



THANK YOU

REFLECTION

“

This project taught me how DDL and DML operations work and the importance of writing queries carefully to manage and maintain data efficiently.

-GUI KAH SIN

”

REFLECTION

“

I realized the importance of enforcing business rules directly in the database (DDL). Using Foreign Keys and Cascades ensures data integrity more reliably than application code.

-SABRINA HENG

”

REFLECTION

“

"Working on advanced DQL showed me that SQL is not just for storage. Using Aggregations and CASE statements allows us to derive powerful insights directly from raw data."

-BRENDAN CHIA

REFLECTION

“

"The optimisation process was eye-opening. Seeing a query drop from 20 row scans to a single lookup using Indexing highlighted the necessity of efficiency in large systems."

-LAU YAN KAI

”